Guia do código — store-microservices

Renato Silva · github.com/renatosilva1983/store-microservices · novembro de 2026

Onde cada recurso do projeto está aplicado, com o arquivo para abrir e o que observar nele. O porquê de cada escolha está nas decisões de arquitetura (ADRs); como rodar, no README.

Os caminhos seguem a mesma estrutura em todos os serviços: `Domain` (regras), `Application` (casos de uso), `Infrastructure` (banco, HTTP, mensageria) e `Api` (controllers e configuração).

Arquitetura

Recurso	Onde está	O que observar
Microsserviços por contexto de negócio	`src/Services`	Catálogo, Pedidos e Identidade, cada um com seu banco (ADR-01, ADR-03)
Clean Architecture	`Ordering`	Quatro projetos por serviço; as referências nos `.csproj` só apontam para dentro (ADR-02)
Composition Root	`Ordering.Api/Program.cs`	Cada camada se registra com uma linha (`AddApplication`, `AddInfrastructure`)
Ports and Adapters	`IOrderRepository.cs` → `OrderRepository.cs`	A Application define a porta; a Infrastructure implementa
Pastas por funcionalidade	`Ordering.Application/Orders`	`Create`, `Cancel`, `Queries`, `Stock`: um caso de uso por pasta
Building blocks	`BuildingBlocks`	O que todos os serviços compartilham, sem regra de negócio
API Gateway (YARP)	`Store.Gateway/appsettings.json`	Rotas e destinos em configuração; o Gateway não referencia nenhum serviço (ADR-04)

Domínio (DDD tático)

Recurso	Onde está	O que observar
Agregado e raiz	`Order.cs`	Itens só mudam pela raiz; invariantes protegidas (pedido cancelado não muda)
Entidade rica	`Product.cs`	Setters privados; métodos de negócio (`ChangePrice`, `DebitStock`)
Value objects	`Money.cs`, `Sku.cs`	Imutáveis, comparados por valor, validados na criação
Factory method	`Order.Create`, `Product.Create`	Só se cria o objeto em estado válido
Cópia de dados entre contextos	`OrderItem.cs`	O pedido guarda nome, SKU e preço do momento da compra

Casos de uso e padrões de projeto

Recurso	Onde está	O que observar
CQRS sem MediatR	`Messaging`	`ICommandHandler` e `IQueryHandler`; leitura e escrita separadas (ADR-12)

Recurso	Onde está	O que observar
Leitura otimizada	`Infrastructure/.../OrderQueries.cs`	`AsNoTracking` e projeção só das colunas usadas
Decorator (validação e log)	`ValidationDecorator.cs`, `LoggingDecorator.cs`, `DependencyInjection.cs`	Envolvem todo handler sem que ele saiba (ADR-13)
Validação	`CreateProductCommandValidator.cs`	FluentValidation, aplicada pelo decorator
Result Pattern	`Result.cs`, `Error.cs`	Erro esperado como valor, sem exceção (ADR-17)
Repository e Unit of Work	`IOrderRepository.cs`	Um repositório por agregado; `SaveChangesAsync` numa transação
Strategy e Factory	`ShippingStrategies.cs`, `ShippingCalculator.cs`	Três regras de frete; regra nova = classe nova; estratégias injetadas como `IEnumerable` (ADR-15)
Adapter / Anti-Corruption Layer	`IProductCatalog.cs` → `CatalogHttpAdapter.cs`	O Pedidos não conhece o formato nem o HTTP do Catálogo (ADR-16)
Options Pattern	`JwtOptions.cs`, `CatalogServiceOptions.cs`	Configuração tipada, validada na subida
Injeção de dependência por camada	`DependencyInjection.cs` de cada projeto	Cada camada registra só o que é dela (ADR-20)

API (MVC)

Recurso	Onde está	O que observar
Controllers finos	`OrdersController.cs`	Traduzem HTTP em comando e resultado em status; ação de negócio como endpoint (`/cancel`)
Result → ProblemDetails	`ApiController.cs`	Tipo de erro vira status HTTP num lugar só
Tratamento global de exceção	`GlobalExceptionHandler.cs`	Sem stack trace na resposta
DTOs de entrada e saída	`OrderRequests.cs`, `ProductResponse.cs`	A API nunca expõe a entidade
GraphQL para leitura	`CatalogQuery.cs`	Resolvers chamam os mesmos handlers do REST; profundidade limitada (ADR-32)
Exemplos de chamada	`Ordering.Api.http`, `Store.Gateway.http`	Roteiros prontos para executar no Visual Studio

Integração entre serviços

Recurso	Onde está	O que observar
HTTP com resiliência (Polly)	`CatalogClientRegistration.cs`	Timeout, retry e circuit breaker; falha vira 503 (ADR-05)
Contratos de integração	`OrderCreatedIntegrationEvent.cs`, `StockIntegrationEvents.cs`	O que os serviços compartilham: só os eventos

Recurso	Onde está	O que observar
Transactional Outbox	`CreateOrderCommandHandler.cs`, `MessagingRegistration.cs`	Evento publicado antes do `SaveChanges`: gravado na mesma transação do pedido (ADR-06)
Consumidor e baixa de estoque	`OrderCreatedConsumer.cs`, `ReserveStockForOrderCommand.cs`	Tudo ou nada: valida todos os itens antes de debitar
Idempotência e consistência eventual	`StockResponseCommands.cs`	Confirmar duas vezes não muda nada; o pedido vai de `Pending` a `Confirmed` ou `Cancelled`
Retry e fila de erro	`MessagingRegistration.cs`	3 novas tentativas, depois a fila `_error`

Segurança

Recurso	Onde está	O que observar
JWT validado em cada serviço	`AuthenticationRegistration.cs`, `TokenService.cs`	Assinatura, emissor, audiência e validade, sem chamar o Identity (ADR-07)
Proteção contra BOLA/IDOR	`OrderAccess.cs`, `ICurrentUser.cs`	O dono vem do token; pedido de outro usuário responde 404 (ADR-42)
Login sem vazar informação	`AuthController.cs`	Comparação em tempo constante; a mesma mensagem para e-mail e senha errados
Rate limiting e headers	`GatewaySecurity.cs`	5 logins por minuto por IP; headers de segurança; sem headers que revelam a tecnologia (ADR-41)
Segurança no pipeline	`ci.yml`, `codeql.yml`, `dependabot.yml`	Dependências vulneráveis quebram o build; CodeQL; Dependabot (ADR-43)

Front-end (Angular 22)

Recurso	Onde está	O que observar
Standalone e zoneless	`app.config.ts`	Sem NgModule e sem zone.js
Signals e Facade	`products.facade.ts`	Estado da tela em `signal` / `computed`; o componente não conhece HTTP
Smart × dumb components	`product-list.ts` × `product-table.ts`	A página coordena; a tabela só recebe `input()` e emite `output()`
RxJS onde faz sentido	`product-list.ts`	`debounceTime` na busca
Formulário reativo tipado	`product-form.ts`	`NonNullableFormBuilder` e validação no TypeScript
Interceptors e guard funcionais	`auth.interceptor.ts`, `error.interceptor.ts`, `auth.guard.ts`	Token anexado, 401 volta ao login, ProblemDetails vira mensagem
Consulta GraphQL	`catalog-api.ts`	Pede só os campos da tabela; confere o array `errors`
Organização por funcionalidade	`src/app`	`core`, `shared` e `features`, como no guia oficial

Testes

Recurso	Onde está	O que observar
Testes de domínio	`OrderTests.cs`, `MoneyTests.cs`	Regras testadas sem mock nem infraestrutura
Casos de uso com mocks	`CreateOrderCommandHandlerTests.cs`	O Catálogo substituído pela porta mockada
Regra de acesso	`OrderAccessTests.cs`	O cliente A não lê nem cancela o pedido do cliente B
Teste de fumaça de segurança	`security-smoke.ps1`	Headers, BOLA/IDOR e rate limiting contra o sistema rodando

Entrega e infraestrutura

Recurso	Onde está	O que observar
Docker multi-stage e compose	`Dockerfile`, `docker-compose.yml`	Sobe bancos, RabbitMQ, serviços e Gateway com um comando
Central Package Management	`Directory.Packages.props`	Todas as versões de pacotes num arquivo só (ADR-34)
CI	`ci.yml`	Build, testes, Angular e Terraform a cada push e PR
Git Flow e versões	fluxo-git.md, CHANGELOG	Branches por funcionalidade, releases com tag e versionamento semântico (ADR-33)
Infraestrutura como código (AWS)	infra/terraform	Rede com subnets isoladas, ECR, RDS, Secrets Manager e deploy por OIDC (ADR-26 a ADR-40)